

CLAVIGER

Rétrospective des interventions humaines ayant orienté la V1.1

Développement assisté par IA - analyse de l'impact du développeur sur la refactorisation,
la maintenabilité et le passage au multi-guild

Objectif du document

Prolonger la rétrospective V1 avec la même méthode : montrer, à partir de la phase V1.1, comment l'assistance IA produit des propositions, des fichiers complets et des tests, tandis que l'humain conserve la direction du produit, challenge les modèles, impose les invariants et décide ce qui devient réellement la nouvelle base du projet.

« Tant pis si c'est plus long [...] on gagnera ça en maintenabilité. »

Extrait utilisateur - phase de refactorisation V1.1

Version de synthèse : V1.1 intermédiaire - 463 tests automatisés verts - readiness par guild intégrée

1. Résumé exécutif

La phase V1.1 confirme et accentue le mode de développement déjà observé en V1 : l'IA produit une part importante de l'implémentation, mais le développeur ne se comporte pas comme un simple validateur final. Il agit comme directeur technique du flux de production : il définit les invariants, refuse des chemins jugés trop fragiles, impose le niveau de maintenabilité, choisit le moment où supprimer la compatibilité et utilise les tests comme portes de passage.

Conclusion en une phrase

L'IA accélère la production du code ; l'humain dirige la transformation du système, fixe les critères d'acceptation, challenge les propositions et décide quand une tranche est suffisamment sûre pour devenir le nouvel état de référence.

- La base de données a été explicitement définie comme propriété de l'application, avec des états indépendants par guild.
- La surface de commandes est désormais construite selon la readiness de chaque guild, afin d'éviter toute fuite de comportement entre serveurs.
- L'identité Discord a été séparée entre application et guild, sans supprimer prématurément le chemin de compatibilité.
- Le développeur a imposé la commande `/bot config-server` sous la racine dynamique et refusé un `/config-server global`.
- La politique de modification a changé : fichier complet lorsque plusieurs zones corrélées changent, plutôt qu'une succession de micro-patches risqués.
- La documentation du code redevient une exigence de production : imports par sections, docstrings de contrat, commentaires sur le pourquoi.
- Les tests restent une gate de livraison : chaque tranche est isolée, validée, puis commitée avant la suivante.

2. Méthode de lecture

Cette rétrospective suit volontairement le même format que le document V1. Une intervention est retenue lorsqu'elle modifie une frontière d'architecture, un invariant de sécurité ou de multi-guild, une règle de maintenabilité, le périmètre d'une tranche, ou la méthode de validation du code produit par l'IA.

Important

Les citations entre guillemets proviennent des échanges de développement accessibles dans cette phase. Les autres fiches sont des synthèses : elles restituent le sens de l'intervention et son effet observable sur le projet, sans prétendre reproduire chaque formulation mot pour mot.

3. Interventions structurantes - chronologie

01

7 septembre 2026
AUDIT

Transformer l'audit en backlog de refactorisation plutôt qu'en justification

Situation initiale - La V1 était fonctionnelle et pouvait être considérée comme suffisamment aboutie pour être simplement déployée.

Intervention humaine - Le développeur demande un audit volontairement critique et accepte que ses constats deviennent une feuille de route de V1.1, notamment sur le multi-guild, le reporting, la sécurité et la documentation.

Effet sur le projet - La documentation cesse d'être décorative : elle devient une source de décisions de production. Les défauts identifiés sont versionnés et traités par tranches.

Trace / indice - La V1.1 naît directement de l'audit pré-déploiement et de ses recommandations prioritaires.

02

7-8 septembre 2026
IDENTITÉ

Séparer identité d'application et identité de guild

Situation initiale - Un seul objet runtime combinait application_id, nom de l'application, bot_user_id, guild_id, nickname et racine de commande.

Intervention humaine - Le développeur valide la séparation conceptuelle : ce qui appartient à l'application ne doit pas être mélangé avec ce qui varie par serveur.

Effet sur le projet - DiscordApplicationIdentity et DiscordGuildIdentity deviennent deux modèles distincts. Le nickname du bot peut varier par guild sans contaminer l'identité globale.

Trace / indice - Refactor validé par tests avant migration du runtime.

03

8 septembre 2026
DONNÉES

Affirmer qu'une base appartient à l'application, pas au serveur

Situation initiale - Le passage au multi-guild aurait pu mener à une base par serveur ou à une initialisation répétée lors de chaque ajout de guild.

Intervention humaine - Le développeur retient explicitement le modèle « 1 application -> 1 DB -> N guilds » et refuse toute réinitialisation de base pour un nouveau serveur.

Effet sur le projet - L'ownership devient application-scoped ; les lignes fonctionnelles restent guild-scoped. Une nouvelle guild réutilise la même base saine.

Trace / indice - Invariant central utilisé ensuite par database ownership et per-guild readiness.

04

8-9 septembre 2026

COMMANDES

Conserver config-server sous la racine dynamique du bot

Situation initiale - Une commande globale /config-server aurait été plus courte à exposer.

Intervention humaine - Le développeur impose /{bot} config-server, pour éviter les collisions et garder l'administration rattachée à l'application qui agit.

Effet sur le projet - Experimentum et Claviger peuvent coexister sur une même guild avec des surfaces d'administration explicitement séparées.

Trace / indice - Convention verrouillée : /experimentum config-server ou /claviger config-server, jamais /config-server.

05

8-9 septembre 2026

ADMIN

Refuser de deviner la sémantique des forums ADMIN

Situation initiale - Une détection par nom, ordre ou position aurait pu attribuer automatiquement activity et errors.

Intervention humaine - Le développeur refuse ce raccourci : l'identité technique vient des IDs ; en cas d'ambiguïté sémantique, l'administrateur doit choisir.

Effet sur le projet - Le pipeline Discovery -> Reconciliation -> Provisioning distingue IMPORT, COMPLETE, CREATE, KEEP et NEEDS_CHOICE sans mutation arbitraire.

Trace / indice - Principe retenu : noms cosmétiques, IDs stables, sémantique explicite.

06

9 septembre 2026

ROBUSTESSE

Préférer la convergence à un rollback Discord naïf

Situation initiale - Une mutation partiellement réussie pouvait pousser à tenter d'annuler automatiquement les changements déjà appliqués.

Intervention humaine - Le développeur privilégie un système capable de redécouvrir l'état réel et de converger au passage suivant, plutôt qu'un rollback incertain sur une API distante.

Effet sur le projet - Le provisioning devient idempotent et tolère mieux les interruptions : un rerun complète ce qui manque au lieu de supposer une transaction Discord parfaite.

Trace / indice - Le scénario « mutation partielle puis rerun » entre dans la campagne de robustesse.

07

10-12 septembre
2026

RUNTIME

Distinguer DB opérationnelle et guild prête

Situation initiale - Une base READY pouvait conduire à exposer les workflows normaux à toutes les guilds de l'application.

Intervention humaine - Le développeur impose une distinction : la santé de la base est globale, mais la configuration ADMIN et donc la readiness sont propres à chaque guild.

Effet sur le projet - Une guild inconnue n'est plus assimilée à une panne DB ; elle reste simplement non configurée et reçoit une surface de récupération/configuration.

Trace / indice - *Invariant formalisé ensuite par GuildConfigurationReadiness.*

08

12 septembre 2026
COMPATIBILITÉ

Ne pas supprimer l'ancien chemin avant que le nouveau soit verrouillé

Situation initiale - Après séparation des identités, il aurait été tentant de supprimer immédiatement DiscordRuntimeIdentity et resolve().

Intervention humaine - Le développeur accepte un pont temporaire et demande de ne démolir l'échafaudage qu'une fois le nouveau runtime couvert par les tests.

Effet sur le projet - Le refactor se fait par migration progressive, réduisant le risque d'un changement structurel trop large.

Trace / indice - *Legacy explicitement conservé comme compatibilité temporaire, avec trajectoire de suppression.*

09

13 septembre 2026
READINESS

Formaliser trois états de readiness par guild

Situation initiale - Un booléen prêt/pas prêt aurait masqué la différence entre absence de configuration et configuration partielle.

Intervention humaine - Le développeur valide READY, ADMIN_CONFIGURATION_MISSING et ADMIN_CONFIGURATION_INCOMPLETE, avec readiness=None quand l'inspection est volontairement impossible.

Effet sur le projet - Le runtime peut expliquer pourquoi une guild n'est pas prête sans confondre nouvelle guild, configuration incomplète et panne applicative.

Trace / indice - *Service dédié GuildConfigurationReadinessService ; absence de row = état normal d'une nouvelle guild.*

10

13 septembre 2026

ISOLATION

Empêcher les workflows d'une guild configurée de fuiter vers une autre

Situation initiale - Avec une DB partagée, une guild B nouvellement rejointe aurait pu hériter implicitement du fait que la DB de l'application est READY.

Intervention humaine - Le développeur exige que membre/noctis et l'admin complet dépendent aussi de la readiness de la guild cible.

Effet sur le projet - Guild A peut être opérationnelle pendant que Guild B ne voit que say + database status + restart + config-server.

Trace / indice - *Test dédié : application saine + guild non configurée = configuration-only mode.*

11

13 septembre 2026

MULTI-GUILD

Découpler l'enregistrement des commandes du legacy self.guild_id

Situation initiale - Le runtime possédait encore un guild_id historique utilisé comme point d'ancrage global.

Intervention humaine - Le développeur valide que _register_guild_commands doit cibler exclusivement la DiscordGuildIdentity fournie.

Effet sur le projet - La primitive de composition sait enregistrer un arbre pour la guild 999 alors que le legacy reste 123. C'est le premier morceau réellement multi-guild.

Trace / indice - *Régression dédiée : commandes présentes uniquement sur la guild explicitement fournie.*

12

13 septembre 2026

TESTS

Utiliser le compteur de tests comme jalon de migration

Situation initiale - Un gros refactor aurait pu rendre difficile l'identification de la tranche responsable d'une régression.

Intervention humaine - Le développeur impose une progression courte : tests ciblés, Ruff, suite complète, puis commit/push avant la tranche suivante.

Effet sur le projet - Les jalons 454, 457, 458, 462 puis 463 tests verts servent de checkpoints reproductibles pendant la refactorisation.

Trace / indice - *Aucun lancement Discord demandé tant que la suite complète n'est pas verte.*

13

13 septembre 2026
MAINTENABILITÉ

Remplacer les micro-patchs corrélés par des fichiers complets

Situation initiale - Le test runtime nécessitait de nombreuses modifications réparties dans le fichier, avec un risque élevé d'oublier une occurrence.

Intervention humaine - Le développeur impose : « même logique hein fichier complet avec explication » dès que plusieurs fonctions ou zones doivent évoluer ensemble.

Effet sur le projet - Le mode de production change : les remplacements cohérents privilégient la lisibilité et réduisent les erreurs de patch manuel.

Trace / indice - Règle désormais intégrée au prompt maître de continuité.

14

13 septembre 2026
DOCUMENTATION

Réintroduire les commentaires comme exigence de production

Situation initiale - À force d'accélérer sur l'architecture, les commentaires et docstrings avaient diminué.

Intervention humaine - Le développeur demande de commenter systématiquement les fonctions : arguments, retour, utilité, exceptions ; imports séparés par sections et commentaires sur le pourquoi.

Effet sur le projet - Les fichiers modifiés deviennent progressivement auto-documentants et pourront servir de base à une documentation technique plus complète.

Trace / indice - Citation : « Ajouter du commentaire un peu partout. »

15

13 septembre 2026
QUALITÉ

Accepter un coût immédiat pour gagner en maintenabilité

Situation initiale - Documenter et renvoyer des fichiers complets augmente le volume de code proposé et ralentit mécaniquement certaines tranches.

Intervention humaine - Le développeur accepte explicitement ce coût : « Tant pis si c'est plus long [...] on gagnera ça en maintenabilité. »

Effet sur le projet - La vitesse de génération n'est plus le critère principal ; la capacité à relire, reprendre et faire évoluer le système devient prioritaire.

Trace / indice - Décision de méthode appliquée immédiatement à `bot.py` et `test_bot.py`.

16

13 septembre 2026
ROADMAP

Refuser de mélanger readiness et lifecycle multi-guild dans une seule tranche

Situation initiale - Il était possible de passer directement à une boucle sur toutes les guilds et aux événements Discord.

Intervention humaine - Le développeur valide une étape intermédiaire : readiness d'abord, puis GuildRuntimeState, puis routine unique par guild, puis lifecycle Discord.

Effet sur le projet - Le chantier multi-guild reste décomposé en responsabilités testables, sans refactor « big bang ».

Trace / indice - Prochaine incision définie : modèle GuildRuntimeState + registre, comportement inchangé.

17

13 septembre 2026
LIFECYCLE

Exiger une vérification du comportement réel de discord.py avant l'orchestration

Situation initiale - setup_hook, on_ready, on_guild_join et on_guild_available ont des timings/cache différents et une supposition erronée pourrait rendre le runtime instable.

Intervention humaine - Le développeur demande que le lifecycle réel de discord.py 2.6 soit vérifié avant de décider quand configurer toutes les guilds.

Effet sur le projet - La prochaine architecture sera fondée sur le comportement réel du framework, pas sur une intuition ou une convention mémorisée.

Trace / indice - Principe : ne pas supposer self.guilds complet dans setup_hook sans vérification.

18

13 septembre 2026
GOUVERNANCE

Documenter explicitement qui produit le code et qui dirige la production

Situation initiale - Une présentation simpliste « humain code / IA assiste » ne décrit plus correctement le workflow observé.

Intervention humaine - Le développeur demande une documentation assumant que l'IA produit une part substantielle du code, tout en montrant que l'utilisateur challenge, redirige et contrôle ce qui est intégré.

Effet sur le projet - La rétrospective devient aussi une trace de l'évolution du métier : moins centré sur la saisie de code, davantage sur modèle mental, arbitrage, validation et direction de production.

Trace / indice - Le présent document prolonge volontairement la nomenclature et la structure de la rétrospective V1.

4. Ce que ces interventions ont changé dans l'architecture V1.1

Avant / risque	Décisions retenues
Runtime centré sur une guild configurée	État applicatif global + readiness indépendante par guild
Identité runtime combinée	DiscordApplicationIdentity + DiscordGuildIdentity
DB READY = commandes normales	DB opérationnelle ET guild prête = commandes normales
Racine admin potentiellement générique	/bot config-server rattaché explicitement à l'application
Détection possible par noms	IDs stables ; noms cosmétiques ; sémantique non devinée
Rollback implicite après mutation partielle	Convergence idempotente à partir de l'état Discord réel
self.guild_id au centre de la registration	Guild identity fournie comme cible explicite
Micro-patches nombreux	Fichier complet lorsque les changements sont corrélés
Commentaires variables	Docstrings de contrat + commentaires sur les invariants et le pourquoi
Refactor multi-guild en bloc	Migration par tranches : readiness -> état runtime -> lifecycle -> cleanup

5. Répartition réelle des responsabilités humain / IA

Comme en V1, l'objectif n'est pas de nier l'assistance IA mais de décrire précisément le partage de responsabilité observé pendant la V1.1.

Assistance IA	Responsabilité humaine observée
Proposer des modèles, services et fichiers complets.	Définir le problème, accepter/refuser le modèle et imposer les invariants.
Écrire une grande partie du code et des tests.	Décider la tranche, exécuter les tests, refuser les régressions et autoriser le commit.
Suggérer des simplifications ou suppressions de legacy.	Choisir le moment où la compatibilité peut réellement être retirée.
Proposer un lifecycle ou une orchestration.	Exiger la vérification du framework réel avant de figer l'architecture.

Assistance IA	Responsabilité humaine observée
Générer des patches locaux.	Imposer un fichier complet lorsque les changements corrélés rendent le patch risqué.
Produire du code fonctionnel.	Exiger commentaires, docstrings, organisation des imports et maintenabilité à long terme.
Analyser les erreurs de tests.	Fournir l'environnement réel, décider du correctif acceptable et vérifier la suite complète.
Proposer la suite technique.	Arbitrer l'ordre : readiness avant lifecycle, multi-guild avant reporting, cleanup après verrouillage.
Aider à documenter le projet.	Assumer publiquement l'usage de l'IA et demander une trace claire de la direction humaine.

Lecture professionnelle

Cette phase montre un déplacement concret du rôle : la valeur du développeur ne se mesure pas seulement au nombre de lignes tapées manuellement. Elle apparaît dans la qualité du modèle mental, la capacité à challenger une proposition générée, à détecter les mauvais invariants, à organiser la migration et à définir les critères permettant d'accepter ou de rejeter la production de l'IA.

La relation observée n'est donc ni « l'IA décide et l'humain approuve », ni « l'humain écrit tout et l'IA complète ». L'IA agit comme producteur technique rapide ; l'humain reste directeur de la production logicielle et responsable du risque accepté.

6. Compétences mises en évidence par les interventions

Compétence	Éléments observés	Interventions
Architecture logicielle	Séparation application/guild, ownership global, readiness locale, migration progressive	2, 3, 7, 8, 9, 16
Multi-guild	Isolation des guilds, registration explicite, futur registre runtime	7, 10, 11, 16, 17
Sécurité / fail-closed	Ownership mismatch, pas de guess sémantique, pas de workflows sur guild non prête	3, 5, 7, 10
Robustesse distribuée	Convergence après mutation Discord partielle, idempotence	6
Maintenabilité	Fichiers complets, docstrings, commentaires, compatibilité temporaire identifiée	8, 13, 14, 15
Qualité logicielle	Tests ciblés, Ruff, suite complète, jalons verts avant commit	12
Gestion de roadmap	Découpage readiness -> runtime state -> lifecycle -> cleanup	1, 16, 17
Développement assisté par IA	Production de code par IA sous challenge, arbitrage et validation humaine	Toutes

7. Corrections secondaires mais révélatrices

Racine dynamique de config-server

Le développeur reprend explicitement la convention `/bot` config-server pour éviter qu'une commande globale soit introduite par facilité lors d'un refactor.

Comptage exact des tests

Lorsque le nombre attendu est discuté, le compteur devient une information de migration : une hausse ou une baisse doit être expliquée par l'ajout ou le retrait réel d'un comportement testé.

Readiness None vs MISSING

Le développeur accepte que `readiness=None` signifie « non inspectée car DB applicative non opérationnelle », distinct de `ADMIN_CONFIGURATION_MISSING`.

Commentaires sur le pourquoi

La documentation demandée ne consiste pas à paraphraser les lignes ; elle doit expliquer arguments, contrat, ordre des opérations, fail-closed et compatibilité.

Séparation des sessions

Quand le contexte devient trop long, un prompt maître de continuité est produit pour transmettre état technique, méthode de travail et roadmap sans perdre les décisions.

8. Ce que ce document permet raisonnablement d'affirmer

Ce qu'il permet d'affirmer

Le développeur a exercé un contrôle continu sur la V1.1 : architecture multi-guild, sémantique des états, sécurité de la migration, convention des commandes, méthode de patch, documentation, séquençage des refactors et validation par tests.

Ce qu'il ne cherche pas à affirmer

Il ne prétend pas que l'humain a écrit chaque ligne. Au contraire, l'IA a produit une part substantielle de l'implémentation. La compétence démontrée ici est la capacité à diriger cette production : comprendre les conséquences, challenger les propositions, imposer les invariants et conserver la responsabilité de l'intégration.

9. Conclusion

La V1.1 rend encore plus visible que la direction humaine ne se limite pas au cadrage initial. Elle apparaît dans les détails mêmes de la migration en cours.

- Le runtime devient multi-guild parce que les états application et guild ont été séparés avant de toucher au lifecycle.
- Une nouvelle guild reste sûre parce que DB opérationnelle et guild readiness sont traitées comme deux dimensions différentes.
- La migration reste maîtrisable parce que la compatibilité n'est supprimée qu'après couverture du nouveau chemin.
- La maintenance future est mieux préparée parce que le développeur a préféré des fichiers complets documentés à une accumulation de patches fragiles.
- La roadmap reste lisible parce que chaque étape est volontairement petite, testée, commitée et poussée avant la suivante.

Mon évaluation : présenter cette phase comme un simple « code généré par IA » serait aussi incomplet que prétendre que l'IA n'a été qu'un autocompléteur. Le code est largement produit avec l'assistance de l'IA ; la direction de production reste humaine. Le rôle du développeur se déplace vers la conception du système, la critique des propositions, l'orchestration du changement, la validation et la responsabilité du résultat intégré.

Annexe - Index rapide des interventions

#	Période	Intervention
1	7 septembre 2026	Transformer l'audit en backlog de refactorisation plutôt qu'en justification
2	7-8 septembre 2026	Séparer identité d'application et identité de guild
3	8 septembre 2026	Affirmer qu'une base appartient à l'application, pas au serveur
4	8-9 septembre 2026	Conserver config-server sous la racine dynamique du bot
5	8-9 septembre 2026	Refuser de deviner la sémantique des forums ADMIN
6	9 septembre 2026	Préférer la convergence à un rollback Discord naïf
7	10-12 septembre 2026	Distinguer DB opérationnelle et guild prête
8	12 septembre 2026	Ne pas supprimer l'ancien chemin avant que le nouveau soit verrouillé
9	13 septembre 2026	Formaliser trois états de readiness par guild

#	Période	Intervention
10	13 septembre 2026	Empêcher les workflows d’une guild configurée de fuiter vers une autre
11	13 septembre 2026	Découpler l’enregistrement des commandes du legacy self.guild_id
12	13 septembre 2026	Utiliser le compteur de tests comme jalon de migration
13	13 septembre 2026	Remplacer les micro-patches corrélés par des fichiers complets
14	13 septembre 2026	Réintroduire les commentaires comme exigence de production
15	13 septembre 2026	Accepter un coût immédiat pour gagner en maintenabilité
16	13 septembre 2026	Refuser de mélanger readiness et lifecycle multi-guild dans une seule tranche
17	13 septembre 2026	Exiger une vérification du comportement réel de discord.py avant l’orchestration
18	13 septembre 2026	Documenter explicitement qui produit le code et qui dirige la production

Document de rétrospective produit à partir de l’historique de développement de la V1.1. Il s’agit d’une analyse qualitative des décisions, corrections et arbitrages humains, et non d’un journal exhaustif ligne par ligne de la génération du code.